

Security & Cryptography

Laboratory Manual

A hands-on introduction to building and breaking ciphers with Python

Course Instructor: Dr. Gehad Taher

Teaching Assistant: Mina Magdy

About this manual

This laboratory manual was prepared for the *Security & Cryptography* course at the Faculty of Computers and Informatics, Suez Canal University. It is intended for classroom use by students of the course.

The labs are inspired by and adapted from *Cracking Codes with Python: An Introduction to Building and Breaking Ciphers* by Al Sweigart (No Starch Press, 2018), which the author released under the [Creative Commons Attribution–NonCommercial–ShareAlike 3.0 United States License](#). The explanations, exercises, and code in this manual have been rewritten and reorganized for laboratory sessions; the original work remains the property of its author. This adapted manual is distributed for non-commercial educational use under the same license.

All code in this manual targets **Python 3**.

Cipher Playground. The course provides an interactive [Cipher Playground](#) where you can try the cipher algorithms directly in your browser — type a plaintext and key, press *Encrypt*, and see the ciphertext. Throughout these labs, after you write a program you can check your result against the matching tool in the Playground. Open it at:

<https://gehadtaher-hub.github.io/Crypto-ebook/studio.html>

Compiled with pdfL^AT_EX.

Contents

1	Setting Up Python and the Caesar Cipher	4
1.1	Setting Up Python	4
1.2	Building the Caesar Cipher	5
1.3	Key Terms	7
2	Monoalphabetic Substitution and Frequency Analysis	8
2.1	Building a Substitution Cipher	8
2.2	Breaking It with Frequency Analysis	10
2.3	Key Terms	10
3	Transposition Ciphers: Rail Fence and Columnar	12
3.1	The Rail Fence Cipher	12
3.2	The Columnar Transposition Cipher	13
3.3	Reversing a Transposition	14
3.4	Key Terms	15
4	The Vigenere Cipher and the One-Time Pad	16
4.1	The Vigenere Cipher	16
4.2	The One-Time Pad	18
4.3	Key Terms	19
5	The Playfair and Hill Ciphers	20
5.1	The Playfair Cipher	20
5.2	The Hill Cipher	22
5.3	Key Terms	24
6	Mathematical Foundations of Cryptography	25
6.1	Modular Exponentiation	25
6.2	The Greatest Common Divisor	26
6.3	The Modular Inverse	27
6.4	Euler's Totient	27
6.5	Key Terms	28
7	Symmetric Encryption: DES and AES	29
7.1	AES in CBC Mode	29
7.2	Why ECB Mode Is Unsafe	30
7.3	The DES Cipher	31
7.4	Key Terms	32
8	Public-Key Cryptography: RSA	33
8.1	Generating a Key Pair	33
8.2	Encrypting and Decrypting a Number	34
8.3	Encrypting Text	35
8.4	Key Terms	36

9	Cryptographic Hash Functions and HMAC	37
9.1	Hashing with SHA-256	37
9.2	The Avalanche Effect	38
9.3	Message Authentication with HMAC	39
9.4	Key Terms	40
A	Instructor Solutions	41

LAB 1

Setting Up Python and the Caesar Cipher

In this first lab you will prepare your Python environment and use it to build your first cipher. We keep the theory short: the concepts behind classical ciphers are covered in the lecture. Here we focus on *doing* — writing, running, and testing real code.

Lab Objectives

- Install Python 3 and verify it from a terminal.
- Run a Python program and read its output.
- Implement the Caesar cipher to encrypt and decrypt messages.
- Break a Caesar-encrypted message by trying every key.

Background *(the full concept is covered in the lecture)*

The Caesar cipher replaces each letter with the one a fixed number of positions later in the alphabet. That fixed number is the secret **key** K . If we number the letters A=0 to Z=25, encryption and decryption are

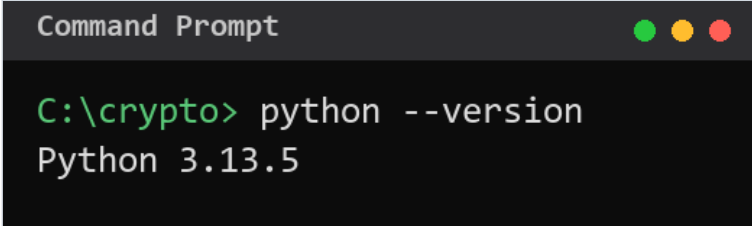
$$C = (P + K) \bmod 26, \quad P = (C - K) \bmod 26.$$

The lecture covers why this cipher is historically important and why it is weak; see the course ebook, Chapter 3. In this lab we implement it in Python.

1.1 Setting Up Python

Every lab in this course uses **Python 3**.

1. Download the latest Python 3 installer from python.org/downloads.
2. Run the installer. On Windows, tick “Add Python to PATH” before clicking *Install Now*.
3. Open a terminal (Command Prompt on Windows) and confirm the installation by asking Python for its version. You should see a version number that starts with 3:



```
Command Prompt
C:\crypto> python --version
Python 3.13.5
```

A successful installation: Python reports its version number.

Note

If the `python` command is not found, close and reopen the terminal, or try `python3 -version`. Save all your course programs in one folder and run them from a terminal opened in that folder.

1.2 Building the Caesar Cipher

We now build the Caesar cipher step by step. Tasks 1.1 and 1.2 are worked out in full so you can follow the method; Task 1.3 we solve together in the lab; Tasks 1.4 and 1.5 are your assignment.

Task 1.1 ■ Worked — Encrypt a message

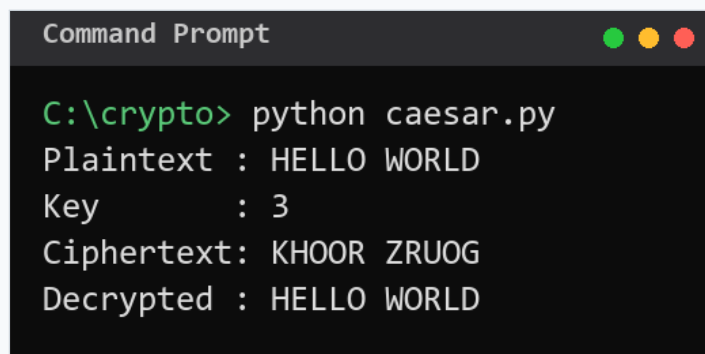
Write a Python program that encrypts the message `HELLO WORLD` with the Caesar cipher using key 3.

Approach. Store the alphabet in a string `SYMBOLS`. For each letter of the message, find its index, add the key, wrap around with `% 26`, and take the letter at the new index. Characters that are not letters (such as the space) are copied unchanged.

Code. Save this as `caesar.py`:

```
caesar.py
1 SYMBOLS = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ'
2
3 def encrypt(message, key):
4     result = ''
5     for symbol in message.upper():
6         if symbol in SYMBOLS:
7             index = SYMBOLS.find(symbol)
8             result = result + SYMBOLS[(index + key) % 26]
9         else:
10            result = result + symbol
11    return result
12
13 message = 'HELLO WORLD'
14 key = 3
15 print('Ciphertext:', encrypt(message, key))
```

Run it and check the output.



```
Command Prompt
C:\crypto> python caesar.py
Plaintext : HELLO WORLD
Key       : 3
Ciphertext: KHOOR ZRUOG
Decrypted : HELLO WORLD
```

Running `caesar.py`. `HELLO WORLD` encrypts to `KHOOR ZRUOG`.

Explanation. The letter H (index 7) becomes index $(7 + 3) \bmod 26 = 10$, which is K; the space is not in `SYMBOLS`, so it is copied through unchanged. (The full `caesar.py` in the

course files also prints the decrypted text, shown above — that is Task 1.2.)

Task 1.2 ■ Worked — Decrypt a message

Extend the program so it can also decrypt, and confirm that decrypting the ciphertext returns the original message.

Approach. Decryption shifts letters *back* by the key. Instead of writing a second loop, notice that shifting back by K is the same as shifting forward by $-K$. So `decrypt` can simply call `encrypt` with a negative key.

Code. Add this function to `caesar.py`:

caesar.py (added)

```
1 def decrypt(message, key):  
2     return encrypt(message, -key)  
3  
4 print('Decrypted :', decrypt('KHOOR ZRUOG', 3))
```

Result. The program prints `Decrypted : HELLO WORLD` (see the last line of the screenshot in Task 1.1). Because `% 26` handles negative numbers correctly in Python, the same alphabet logic works in both directions.

Cipher Playground — Caesar Cipher

Check your program against the interactive tool. Open the [Cipher Playground](#), find the **Caesar Cipher** card, type the same plaintext (for example `HELLO WORLD`) and the same key (3), and press *Encrypt*. The ciphertext it shows should match your program's output, `KHOOR ZRUOG`, exactly. Try a few different keys and confirm your program agrees every time.

Task 1.3 ■ In-Lab — Break it by brute force

The Caesar cipher has only 25 useful keys, so an attacker can try them all. You intercept the ciphertext `KHOOR ZRUOG` but do *not* know the key.

Write a program `caesarBrute.py` that decrypts the ciphertext with every key from 1 to 25 and prints all 25 candidate messages. Then read the list and identify which key was used. We will build and run this together in the lab.

Task 1.4 ■ Assignment — Encrypt your own input

Modify `caesar.py` so that, instead of a fixed message and key, it asks the user for a message and a key with `input()`, then prints the ciphertext. Remember that `input()` returns text, so the key must be converted to an integer with `int(...)`. Test it by encrypting your own name with key 7.

Task 1.5 ■ Assignment — Crack an unknown message

A classmate sends you the ciphertext `WKLV LV IXQ`, encrypted with the Caesar cipher, but does not tell you the key. Use your brute-force program from Task 1.3 to recover the original message, and state which key was used.

1.3 Key Terms

Key Terms

Key The secret value that selects the shift in the Caesar cipher.

Encrypt / Decrypt Turning plaintext into ciphertext, and back again.

Modulo (%) The remainder after division; makes the alphabet wrap around.

Brute-force attack Trying every possible key until the message is readable.

LAB 2

Monoalphabetic Substitution and Frequency Analysis

The Caesar cipher was easy to break because it had only 25 keys. In this lab you build a much stronger classical cipher — the monoalphabetic substitution cipher — and then discover the clever technique that still breaks it: frequency analysis. As always, the theory is in the lecture; here we build and run the code.

Lab Objectives

- Implement a monoalphabetic substitution cipher in Python.
- Encrypt and decrypt messages using a scrambled-alphabet key.
- Count letter frequencies in a ciphertext with a program.
- Use frequency analysis to begin breaking a substitution cipher.

Background *(the full concept is covered in the lecture)*

A **monoalphabetic substitution cipher** replaces each letter with a different fixed letter. The key is a scrambled version of the alphabet: the first letter says what A becomes, the second what B becomes, and so on. There are 26! possible keys (more than 4×10^{26}), so trying them all is hopeless. Yet the cipher has a fatal flaw: because each letter always maps to the same symbol, the *frequencies* of the letters are preserved. Since E is the most common letter in English, the most common letter in the ciphertext is probably E. Exploiting this is called **frequency analysis** (see the course ebook, Chapter 3).

2.1 Building a Substitution Cipher

Tasks 2.1 and 2.2 are worked out in full; Task 2.3 we solve together in the lab; Tasks 2.4 and 2.5 are your assignment.

Task 2.1 ■ Worked — Encrypt with a substitution key

Encrypt HELLO WORLD using a scrambled alphabet as the key.

Approach. Keep the normal alphabet in `SYMBOLS` and the scrambled alphabet in `KEY`. To encrypt a letter, find its position in `SYMBOLS` and output the letter at the same position in `KEY`.

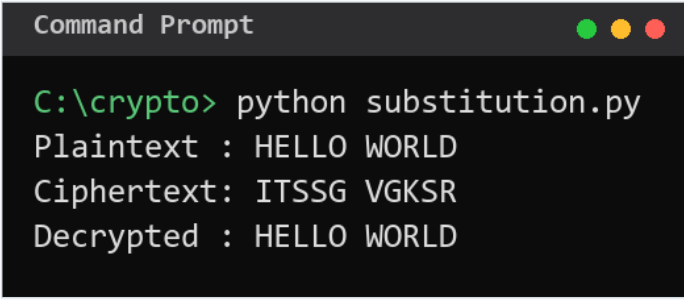
Code. Save this as `substitution.py`:

substitution.py

```
1 SYMBOLS = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ'  
2 KEY     = 'QWERTYUIOPASDFGHJKLZXCVBNM'
```

```
3
4 def encrypt(message):
5     result = ''
6     for symbol in message.upper():
7         if symbol in SYMBOLS:
8             result = result + KEY[SYMBOLS.find(symbol)]
9         else:
10            result = result + symbol
11    return result
12
13 print('Ciphertext:', encrypt('HELLO WORLD'))
```

Run it and check the output.



```
Command Prompt

C:\crypto> python substitution.py
Plaintext : HELLO WORLD
Ciphertext: ITSSG VGKSR
Decrypted : HELLO WORLD
```

HELLO WORLD encrypts to ITSSG VGKSR under this key.

Explanation. H is at position 7 of SYMBOLS, and position 7 of KEY is I, so H becomes I. Every L becomes the same letter S — that repetition is exactly what frequency analysis will later exploit.

Task 2.2 ■ Worked — Decrypt with the same key

Add decryption and confirm the ciphertext turns back into the plaintext.

Approach. Decryption is the mirror image: find the letter in KEY and output the letter at the same position in SYMBOLS.

Code. Add this to substitution.py:

substitution.py (added)

```
1 def decrypt(message):
2     result = ''
3     for symbol in message.upper():
4         if symbol in KEY:
5             result = result + SYMBOLS[KEY.find(symbol)]
6         else:
7             result = result + symbol
8     return result
9
10 print('Decrypted :', decrypt('ITSSG VGKSR'))
```

Result. The program prints Decrypted : HELLO WORLD (shown in the screenshot above). Encryption and decryption use the same key; this is a symmetric cipher.

2.2 Breaking It with Frequency Analysis

Brute force cannot try $26!$ keys, but it does not need to. Because letter frequencies survive encryption, we can count how often each ciphertext letter appears and match the most common ones to the most common English letters (E, T, A, O, I, N, ...).

Task 2.3 ■ In-Lab — Count letter frequencies

Write a program `frequency.py` that counts how often each letter appears in the ciphertext below and prints the letters from most to least frequent.

```
ZIT STZZTK T OL ZIT DGLZ EGDDGF STZZTK OF TFUSOLI ZTBZ
QFR YKTJXFEN QFQSNLLOL XLTL ZIOL YQEZ
```

Which ciphertext letter is the most frequent, and which plaintext letter does it most likely represent? We will write and run this together in the lab.

Cipher Playground — Frequency Analyzer

You can check your letter counts with the interactive tool. Open the [Cipher Playground](#), find the **Frequency Analyzer** card, paste the same ciphertext, and press *Analyze*. Compare its counts with the output of your `frequency.py` program.

Task 2.4 ■ Assignment — Implement the Atbash cipher

The **Atbash cipher** is a simple fixed substitution: each letter maps to its mirror, so $A \leftrightarrow Z$, $B \leftrightarrow Y$, and so on. Write a program `atbash.py` that encrypts a message with Atbash. *Hint*: the mirror of the letter at index i is at index $25 - i$. Encrypt HELLO WORLD and confirm you get SVOOL DLIWL.

Cipher Playground — Atbash Cipher

Verify your Atbash program: open the [Cipher Playground](#), find the **Atbash Cipher** card, enter HELLO WORLD, and press *Transform*. The result should match your program's output exactly.

Task 2.5 ■ Assignment — Reason about the frequencies

Run your `frequency.py` on the ciphertext from Task 2.3. The two most frequent ciphertext letters are T and Z. Assuming they correspond to the two most common English letters, state which plaintext letter each one most likely represents, and explain your reasoning in one or two sentences.

2.3 Key Terms

Key Terms

Monoalphabetic substitution A cipher that replaces each letter with one fixed other letter.

Key (substitution) The scrambled alphabet that defines every letter's replacement.

Frequency analysis Breaking a cipher by matching common ciphertext letters to common language letters.

Atbash A fixed substitution in which each letter maps to its mirror in the alphabet.

LAB 3

Transposition Ciphers: Rail Fence and Columnar

So far our ciphers have *substituted* letters. Transposition ciphers do something different: they keep the same letters but *rearrange* their order. In this lab you implement two classic transposition ciphers — the Rail Fence and the Columnar Transposition — and check your programs against the Cipher Playground.

Lab Objectives

- Explain how a transposition cipher differs from a substitution cipher.
- Implement the Rail Fence cipher in Python.
- Implement the Columnar Transposition cipher using a keyword.
- Recover a plaintext by reversing the Rail Fence.

Background *(the full concept is covered in the lecture)*

A **transposition cipher** scrambles the *positions* of the letters instead of replacing them. The **Rail Fence** cipher writes the message in a zigzag across a number of “rails” and then reads it off row by row. The **Columnar Transposition** cipher writes the message in rows under a keyword, then reads the columns in the alphabetical order of the keyword’s letters. Because the letters themselves are unchanged, frequency analysis alone does not break these ciphers — the lecture explains why (course ebook, Chapter 3).

3.1 The Rail Fence Cipher

Imagine writing the message diagonally down and up across a set of rails. With 3 rails, the word WEAREDISCOVERED is placed like this:

```
W . . . E . . . C . . . R . .  
. E . R . D . S . O . E . E .  
. . A . . . I . . . V . . . D
```

Reading the rails top to bottom gives the ciphertext.

Task 3.1 ■ Worked — Encrypt with the Rail Fence

Encrypt WEAREDISCOVERED using 3 rails.

Approach. Keep one string per rail. Walk through the message moving down the rails and then back up, appending each letter to the current rail. Finally join the rails together.

Code. Save this as `railfence.py`:

railfence.py

```

1 def encrypt(message, rails):
2     fence = ['' for _ in range(rails)]
3     row = 0
4     step = 1
5     for symbol in message:
6         fence[row] = fence[row] + symbol
7         if row == 0:
8             step = 1
9         elif row == rails - 1:
10            step = -1
11        row = row + step
12    return ''.join(fence)
13
14 print('Ciphertext:', encrypt('WEAREDISCOVERED', 3))

```

Run it and check the output.

```

Command Prompt

C:\crypto> python railfence.py
Plaintext : WEAREDISCOVERED
Rails      : 3
Ciphertext: WECRERDSOEEAIVD

```

WEAREDISCOVERED with 3 rails encrypts to WECRERDSOEEAIVD.

Explanation. The variable `step` flips between `+1` and `-1` at the top and bottom rails, producing the zigzag. Joining rail 0 (WECR), rail 1 (ERDSOEE), and rail 2 (AIVD) gives the ciphertext.

Cipher Playground — Rail Fence Cipher

Verify your program: open the [Cipher Playground](#), find the **Rail Fence Cipher** card, enter WEAREDISCOVERED with 3 rails, and press *Encrypt*. It should show WECRERDSOEEAIVD, matching your output.

3.2 The Columnar Transposition Cipher

Here we write the message in rows whose width equals the length of a keyword, then read the columns in the alphabetical order of the keyword's letters. For MEETMEAFETERCLASS under the keyword ZEBRA:

Z	E	B	R	A
M	E	E	T	M
E	A	F	T	E
R	C	L	A	S
S				

The keyword letters in alphabetical order are A B E R Z, so we read the columns in that order.

Task 3.2 ■ Worked — Encrypt with Columnar Transposition

Encrypt MEETMEAFETERCLASS using the keyword ZEBRA.

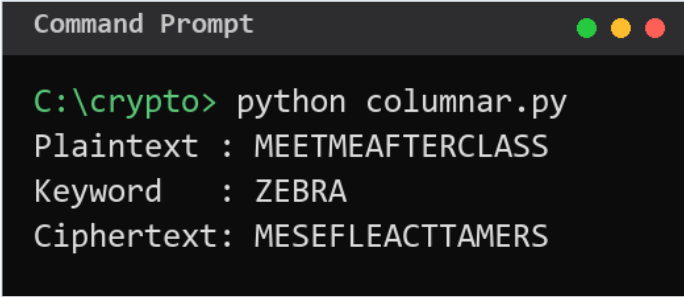
Approach. Spread the letters across columns (letter i goes to column $i \% \text{len}(\text{key})$). Then decide the reading order by sorting the column indexes according to the keyword's letters, and join the columns in that order.

Code. Save this as `columnar.py`:

```
columnar.py

1 def encrypt(message, key):
2     columns = ['' for _ in range(len(key))]
3     for i, symbol in enumerate(message):
4         columns[i % len(key)] = columns[i % len(key)] + symbol
5     order = sorted(range(len(key)), key=lambda i: (key[i], i))
6     return ''.join(columns[i] for i in order)
7
8 print('Ciphertext:', encrypt('MEETMEAFETERCLASS', 'ZEBRA'))
```

Run it and check the output.



```
Command Prompt

C:\crypto> python columnar.py
Plaintext : MEETMEAFETERCLASS
Keyword   : ZEBRA
Ciphertext: MESEFLEACTTAMERS
```

MEETMEAFETERCLASS under keyword ZEBRA encrypts to MESEFLEACTTAMERS.

Explanation. Column A (MES) is read first, then B (EFL), E (EAC), R (TTA), and Z (MERS). Joining them gives MESEFLEACTTAMERS.

Cipher Playground — Columnar Transposition

Check it in the [Cipher Playground](#): the **Columnar Transposition** card, message MEETMEAFETERCLASS, keyword ZEBRA, *Encrypt*. The result should be MESEFLEACTTAMERS.

3.3 Reversing a Transposition

Task 3.3 ■ In-Lab — Decrypt the Rail Fence

You receive the Rail Fence ciphertext WECRERDSOEAAIVD, encrypted with 3 rails. Write a program `railfenceDecrypt.py` that recovers the original message. *Hint:* first work out how many letters land on each rail by walking the zigzag; cut the ciphertext into those rails; then walk the zigzag again, taking the next letter from the correct rail each time. We will build this together in the lab.

Task 3.4 ■ Assignment — Rail Fence with more rails

Using your `railfence.py`, encrypt `WEAREDISCOVERED` again but with 4 rails instead of 3. Write down the new ciphertext, verify it in the Cipher Playground (Rail Fence card, rails = 4), and explain in one sentence why changing the number of rails changes the output.

Task 3.5 ■ Assignment — Columnar with your own keyword

Use `columnar.py` to encrypt the message `ATTACKATDAWN` with a keyword of your choice (at least 4 letters). Verify your ciphertext against the Columnar Transposition tool in the [Cipher Playground](#) using the same message and keyword.

3.4 Key Terms

Key Terms

Transposition cipher A cipher that rearranges the positions of the letters instead of substituting them.

Rail Fence A transposition that writes the message in a zigzag across several rails.

Columnar Transposition A transposition that reads columns of a grid in the alphabetical order of a keyword.

Keyword The word whose letter order sets the column reading order.

LAB 4

The Vigenere Cipher and the One-Time Pad

The substitution cipher of Lab 2 fell to frequency analysis because every plaintext letter always mapped to the same ciphertext letter. The Vigenere cipher fixes this by using several shifts, driven by a keyword. Push that idea to its limit — a random key as long as the message — and you reach the one-time pad, the only cipher that is provably unbreakable.

Lab Objectives

- Implement the Vigenere cipher to encrypt and decrypt with a keyword.
- Explain how a keyword produces several different shifts.
- Implement a one-time pad and see why the key must equal the message length.
- Relate the one-time pad to the modern XOR form.

Background *(the full concept is covered in the lecture)*

The **Vigenere cipher** is a polyalphabetic cipher: it applies a Caesar shift to each letter, but the size of the shift is chosen by the corresponding letter of a repeating **keyword**. Ciphertext letter C_i is $(P_i + K_i) \bmod 26$, where K_i is the keyword letter above position i . Because the same plaintext letter can now encrypt to different ciphertext letters, simple frequency analysis no longer works. The **one-time pad** is the extreme case: a key that is truly random, as long as the message, and never reused. The lecture explains why that gives perfect secrecy (course ebook, Chapter 3).

4.1 The Vigenere Cipher

Tasks 4.1 and 4.2 are worked; Task 4.3 is solved in the lab; Tasks 4.4 and 4.5 are your assignment.

Task 4.1 ■ Worked — Encrypt with Vigenere

Encrypt ATTACKATDAWN with the keyword LEMON.

Approach. Repeat the keyword across the message. For each letter, the shift is the position of the keyword letter (A=0, B=1, ...). Add that shift with % 26, exactly like a Caesar cipher but with a shift that changes each letter.

Code. Save this as `vigenere.py`:

`vigenere.py`

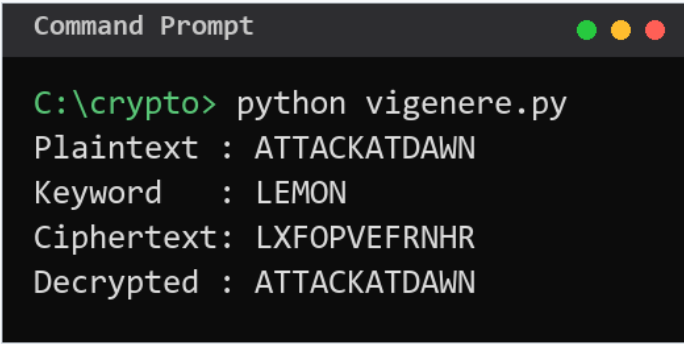
```
1 SYMBOLS = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ'
2
3 def encrypt(message, key):
```

```

4     result = ''
5     key = key.upper()
6     k = 0
7     for symbol in message.upper():
8         if symbol in SYMBOLS:
9             shift = SYMBOLS.find(key[k % len(key)])
10            result = result + SYMBOLS[(SYMBOLS.find(symbol) + shift) % 26]
11            k = k + 1
12        else:
13            result = result + symbol
14    return result
15
16    print('Ciphertext:', encrypt('ATTACKATDAWN', 'LEMON'))

```

Run it and check the output.



```

Command Prompt

C:\crypto> python vigenere.py
Plaintext : ATTACKATDAWN
Keyword   : LEMON
Ciphertext: LXFOPVEFRNHR
Decrypted : ATTACKATDAWN

```

ATTACKATDAWN with keyword LEMON encrypts to LXFOPVEFRNHR.

Explanation. The keyword repeats as LEMONLEMONLE. The first A is shifted by L (11) to give L; the first T is shifted by E (4) to give X; and so on. The two Ts at the start encrypt to different letters, which is exactly what defeats frequency analysis.

Cipher Playground — Vigenère Cipher

Verify your program: open the [Cipher Playground](#), find the **Vigenère Cipher** card, enter ATTACKATDAWN with keyword LEMON, and press *Encrypt*. It should show LXFOPVEFRNHR.

Task 4.2 ■ Worked — Decrypt with Vigenere

Recover the plaintext from LXFOPVEFRNHR using the same keyword.

Approach. Decryption subtracts each keyword shift instead of adding it. Everything else is identical.

Code. Add this to vigenere.py:

vigenere.py (added)

```

1     def decrypt(message, key):
2         result = ''
3         key = key.upper()
4         k = 0
5         for symbol in message.upper():
6             if symbol in SYMBOLS:
7                 shift = SYMBOLS.find(key[k % len(key)])
8                 result = result + SYMBOLS[(SYMBOLS.find(symbol) - shift) % 26]

```

```
9         k = k + 1
10     else:
11         result = result + symbol
12     return result
13
14 print('Decrypted :', decrypt('LXFOPVEFRNHR', 'LEMON'))
```

Result. The program prints `Decrypted : ATTACKATDAWN` (shown in the screenshot above). Encryption adds the keyword shifts; decryption subtracts them.

4.2 The One-Time Pad

If the keyword is as long as the message, truly random, and never reused, the Vigenere cipher becomes the **one-time pad** — the only cipher that cannot be broken, because every possible plaintext is equally consistent with the ciphertext.

Task 4.3 ■ In-Lab — Build a one-time pad

Write a program `otp.py` that encrypts a message with a one-time pad. It should generate a random key the same length as the message (each key letter chosen from A–Z), encrypt with $C_i = (P_i + K_i) \bmod 26$, and then decrypt to confirm it recovers the original. We will build and run this together in the lab.

Task 4.4 ■ Assignment — Vigenere with your own keyword

Use `vigenere.py` to encrypt MEETMEATNOON with a keyword of your choice (at least 3 letters). Verify your ciphertext in the [Cipher Playground](#) using the Vigenère card with the same message and keyword, then decrypt it back to check you recover the message.

Task 4.5 ■ Assignment — The XOR one-time pad

Modern one-time pads work on bits, not letters, using the XOR operation. Write a program `xorpad.py` that XORs each character of a message with a repeating key and prints the result as hexadecimal bytes. Test it with the message HELLO and key KEY. Verify your output against the **XOR Demonstrator** in the [Cipher Playground](#) (it should be 03 00 15 07 0a). In one sentence, explain why reusing an XOR key for two messages is dangerous.

Cipher Playground — XOR Demonstrator

The [Cipher Playground](#) has an **XOR Demonstrator** card. Enter the message HELLO and key KEY, press *Apply XOR*, and compare its hexadecimal output with your `xorpad.py` program.

4.3 Key Terms

Key Terms

Vigenere cipher A polyalphabetic cipher that shifts each letter by a repeating keyword.

Polyalphabetic Using several substitution alphabets so one letter can encrypt many ways.

One-time pad A cipher whose key is random, as long as the message, and used only once.

Perfect secrecy The property that the ciphertext reveals nothing about the plaintext.

XOR The bitwise exclusive-or operation used by modern stream ciphers and pads.

LAB 5

The Playfair and Hill Ciphers

The classical ciphers so far worked on one letter at a time. The two ciphers in this lab are stronger because they encrypt letters in *groups*: Playfair works on pairs of letters using a key square, and Hill works on blocks of letters using matrix multiplication. Both resist simple frequency analysis.

Lab Objectives

- Build a Playfair key square from a keyword and encrypt with it.
- Apply the Playfair pair rules and reverse them to decrypt.
- Implement the Hill cipher using modular matrix multiplication.

Background *(the full concept is covered in the lecture)*

The **Playfair cipher** arranges 25 letters (I and J share a cell) into a 5×5 square built from a keyword, and encrypts *pairs* of letters using three rules based on whether the pair shares a row, shares a column, or forms a rectangle. The **Hill cipher** treats each block of letters as a vector of numbers and multiplies it by a key matrix, all modulo 26. The lecture covers the full theory (course ebook, Chapter 3); here we implement both.

Note

The Playfair and Hill ciphers are not in the Cipher Playground, so for this lab you verify your work a different way: encrypt a message, then *decrypt the result* and check you get the original message back.

5.1 The Playfair Cipher

To encrypt, split the message into pairs of letters and apply these rules to each pair, using the key square:

- **Same row:** replace each letter with the one to its right (wrapping around).
- **Same column:** replace each letter with the one below it (wrapping around).
- **Rectangle:** replace each letter with the one in its own row but the other letter's column.

A doubled pair (like LL) is split by inserting X, and a leftover final letter is padded with X.

Task 5.1 ■ Worked — Build the square and encrypt

Encrypt HIDE THE GOLD with the keyword MONARCHY.

Approach. First build the key square: write the keyword's letters (dropping repeats), then fill in the rest of the alphabet. For MONARCHY the square reads MONARCHYBDEFGIKLPQSTUVWXZ. Then split the message into pairs and apply the three rules.

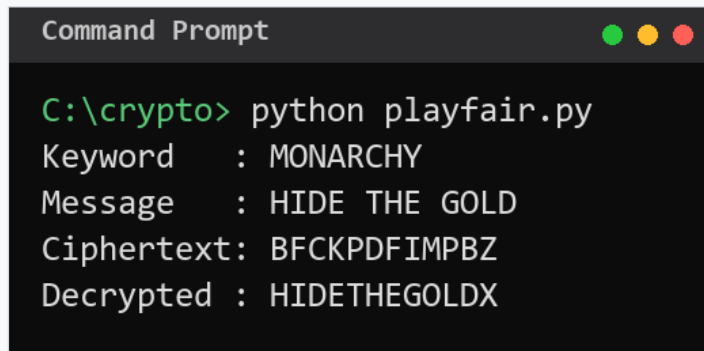
Code. Save this as `playfair.py`:

```

                                playfair.py
1  ALPHABET = 'ABCDEFGHIKLMNOPQRSTUVWXYZ'  # no J
2
3  def build_square(keyword):
4      seen = ''
5      for ch in keyword.upper().replace('J', 'I') + ALPHABET:
6          if ch in ALPHABET and ch not in seen:
7              seen = seen + ch
8      return seen
9
10 def make_pairs(text):
11     text = ''.join(c for c in text.upper() if c.isalpha()).replace('J', 'I')
12     pairs, i = [], 0
13     while i < len(text):
14         a = text[i]
15         b = text[i + 1] if i + 1 < len(text) else 'X'
16         if a == b:
17             pairs.append(a + 'X'); i = i + 1
18         else:
19             pairs.append(a + b); i = i + 2
20     return pairs
21
22 def transform(text, keyword, direction):
23     square = build_square(keyword)
24     result = ''
25     for pair in make_pairs(text):
26         i1 = square.find(pair[0]); r1, c1 = i1 // 5, i1 % 5
27         i2 = square.find(pair[1]); r2, c2 = i2 // 5, i2 % 5
28         if r1 == r2:
29             result += square[r1 * 5 + (c1 + direction) % 5]
30             result += square[r2 * 5 + (c2 + direction) % 5]
31         elif c1 == c2:
32             result += square[((r1 + direction) % 5) * 5 + c1]
33             result += square[((r2 + direction) % 5) * 5 + c2]
34         else:
35             result += square[r1 * 5 + c2] + square[r2 * 5 + c1]
36     return result
37
38 def encrypt(text, keyword):
39     return transform(text, keyword, +1)
40
41 print('Ciphertext:', encrypt('HIDE THE GOLD', 'MONARCHY'))

```

Run it and check the output.



```

Command Prompt

C:\crypto> python playfair.py
Keyword   : MONARCHY
Message   : HIDE THE GOLD
Ciphertext: BFCKPDFIMPBZ
Decrypted : HIDETHEGOLDX

```

HIDE THE GOLD with keyword MONARCHY encrypts to BFCKPDFIMPBZ.

Explanation. The message becomes the pairs HI DE TH EG OL DX (an X pads the final D). Each pair is replaced using the rules — for example HI forms a rectangle and becomes BF.

Task 5.2 ■ Worked — Decrypt to verify

Recover the message from BFCKPDFIMPBZ.

Approach. Decryption uses the same three rules but moves the other way — left instead of right, and up instead of down. Our `transform` function already supports this with `direction = -1`.

Code. Add:

playfair.py (added)

```

1 def decrypt(text, keyword):
2     return transform(text, keyword, -1)
3
4 print('Decrypted :', decrypt('BFCKPDFIMPBZ', 'MONARCHY'))

```

Result. The program prints `Decrypted : HIDETHEGOLDX`, which is the original message (with the padding X still attached). Recovering the plaintext this way is how you verify a Playfair program.

Task 5.3 ■ In-Lab — Encrypt a new message

Using `playfair.py`, encrypt the message MEET ME AT NOON with the keyword SECURITY. Write down the key square you get, then the ciphertext, and confirm by decrypting it back. We will do this together in the lab.

Task 5.4 ■ Assignment — Playfair with your own keyword

Choose your own keyword and a short message, and use `playfair.py` to encrypt it. Then decrypt the ciphertext and confirm you recover your message (remember it may end with a padding X). Hand in the keyword, the message, the ciphertext, and the decrypted result.

5.2 The Hill Cipher

The Hill cipher turns each pair of letters into a pair of numbers ($A=0$ to $Z=25$), multiplies that vector by a 2×2 key matrix, and reduces modulo 26. With the key matrix $\begin{bmatrix} 3 & 3 \\ 2 & 5 \end{bmatrix}$, a pair (x_0, x_1)

becomes

$$c_0 = (3x_0 + 3x_1) \bmod 26, \quad c_1 = (2x_0 + 5x_1) \bmod 26.$$

Task 5.5 ■ Worked — Encrypt with a key matrix

Encrypt **HELP** with the key matrix above.

Approach. Split the message into pairs of letters, convert each letter to a number, apply the two formulas, and convert the results back to letters.

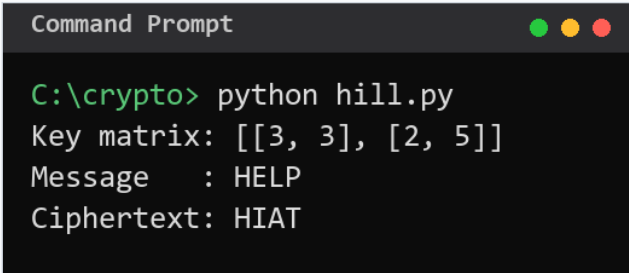
Code. Save this as `hill.py`:

```

hill.py
1 KEY = [[3, 3],
2       [2, 5]]
3
4 def encrypt(message, key):
5     text = ''.join(c for c in message.upper() if c.isalpha())
6     if len(text) % 2 == 1:
7         text = text + 'X'
8     result = ''
9     for i in range(0, len(text), 2):
10        x0 = ord(text[i]) - 65
11        x1 = ord(text[i + 1]) - 65
12        c0 = (key[0][0] * x0 + key[0][1] * x1) % 26
13        c1 = (key[1][0] * x0 + key[1][1] * x1) % 26
14        result = result + chr(c0 + 65) + chr(c1 + 65)
15    return result
16
17 print('Ciphertext:', encrypt('HELP', KEY))

```

Run it and check the output.



```

Command Prompt
C:\crypto> python hill.py
Key matrix: [[3, 3], [2, 5]]
Message : HELP
Ciphertext: HIAT

```

HELP encrypts to HIAT with this key matrix.

Explanation. The pair **HE** is (7,4), giving $c_0 = (3 \cdot 7 + 3 \cdot 4) \bmod 26 = 33 \bmod 26 = 7 = \text{H}$ and $c_1 = (2 \cdot 7 + 5 \cdot 4) \bmod 26 = 34 \bmod 26 = 8 = \text{I}$, so **HE** becomes **HI**. The pair **LP** becomes **AT**.

Task 5.6 ■ Assignment — Hill with your own message

Use `hill.py` to encrypt a message of your choice (letters only) with the same key matrix. Decryption requires the *inverse* of the key matrix modulo 26 — we will learn how to compute modular inverses in Lab 6, so for now just report your plaintext and the ciphertext your program produces.

5.3 Key Terms

Key Terms

Playfair cipher A cipher that encrypts pairs of letters using a 5×5 key square.

Key square The 5×5 arrangement of letters built from the Playfair keyword.

Digraph A pair of letters; Playfair encrypts one digraph at a time.

Hill cipher A cipher that encrypts blocks of letters by matrix multiplication modulo 26.

Key matrix The square matrix of numbers that the Hill cipher multiplies each block by.

LAB 6

Mathematical Foundations of Cryptography

Modern public-key cryptography, such as RSA, is built on a small set of ideas from number theory: modular arithmetic, the greatest common divisor, modular inverses, and Euler's totient. In this lab you implement each one in Python. These are the exact tools you will use to build RSA in Lab 8.

Lab Objectives

- Compute modular exponentiation efficiently in Python.
- Find the greatest common divisor with the Euclidean algorithm.
- Compute modular inverses with the Extended Euclidean algorithm.
- Compute Euler's totient and use these tools to prepare RSA keys.

Background *(the full concept is covered in the lecture)*

Working **modulo** m means keeping only the remainder after dividing by m . Two numbers are **relatively prime** (coprime) when their **greatest common divisor** (GCD) is 1. The **modular inverse** of a modulo m is the number x with $(a \cdot x) \bmod m = 1$; it exists only when a and m are coprime. **Euler's totient** $\varphi(n)$ counts how many numbers from 1 to n are coprime to n . The lecture develops the theory (course ebook, Chapter 4); here we turn each idea into code.

6.1 Modular Exponentiation

Task 6.1 ■ Worked — Compute a modular power

Compute $3^{13} \bmod 17$.

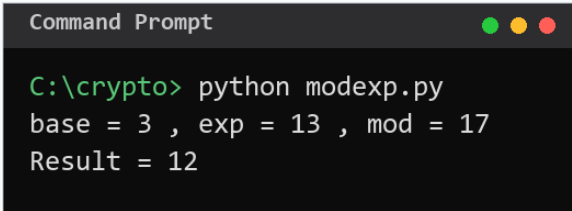
Approach. Raising to a power and then taking a remainder would create huge numbers. Python's built-in `pow(base, exp, mod)` does it efficiently, reducing at every step.

Code. Save this as `modexp.py`:

modexp.py

```
1 base = 3
2 exp = 13
3 mod = 17
4 print('Result =', pow(base, exp, mod))
```

Run it and check the output.

A screenshot of a Windows Command Prompt window. The title bar says "Command Prompt". The prompt is "C:\crypto> python modexp.py". The output is "base = 3 , exp = 13 , mod = 17" followed by "Result = 12".

```
C:\crypto> python modexp.py
base = 3 , exp = 13 , mod = 17
Result = 12
```

$$3^{13} \bmod 17 = 12.$$

Explanation. `pow` with three arguments computes $(\text{base}^{\text{exp}}) \bmod \text{mod}$ without ever building the giant number 3^{13} . This is the operation at the heart of RSA encryption.

Cipher Playground — Modular Power

Verify it: open the [Cipher Playground](#), go to **Crypto Calculations**, and use the **Modular Power** tool with base 3, exponent 13, modulus 17. It should show **Result** = 12.

6.2 The Greatest Common Divisor

Task 6.2 ■ Worked — Find a GCD

Compute $\text{gcd}(252, 105)$.

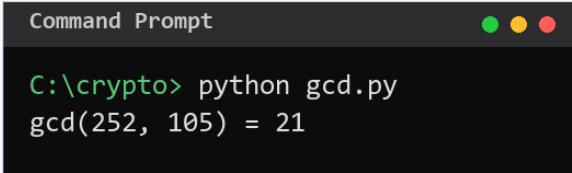
Approach. The Euclidean algorithm repeatedly replaces the pair (a, b) with $(b, a \bmod b)$ until the second number is 0; the first number is then the GCD.

Code. Save this as `gcd.py`:

gcd.py

```
1 def gcd(a, b):
2     while b != 0:
3         a, b = b, a % b
4     return a
5
6 print('gcd(252, 105) =', gcd(252, 105))
```

Run it and check the output.

A screenshot of a Windows Command Prompt window. The title bar says "Command Prompt". The prompt is "C:\crypto> python gcd.py". The output is "gcd(252, 105) = 21".

```
C:\crypto> python gcd.py
gcd(252, 105) = 21
```

$$\text{gcd}(252, 105) = 21.$$

Explanation. The steps are $\text{gcd}(252, 105) \rightarrow \text{gcd}(105, 42) \rightarrow \text{gcd}(42, 21) \rightarrow \text{gcd}(21, 0) = 21$. Because the GCD is not 1, the numbers are *not* coprime.

Cipher Playground — GCD

Check it in the [Cipher Playground](#) under **Crypto Calculations**: the **GCD** tool with 252 and 105 gives **GCD** = 21.

6.3 The Modular Inverse

Task 6.3 ■ Worked — Compute a modular inverse

Find the inverse of 3 modulo 26 (the number x with $(3x) \bmod 26 = 1$).

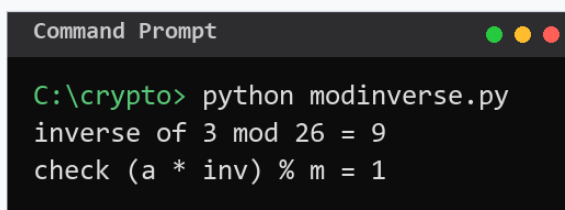
Approach. The Extended Euclidean algorithm returns not only the GCD but also numbers x, y with $ax + my = \gcd(a, m)$. When the GCD is 1, that x (reduced modulo m) is the modular inverse.

Code. Save this as `modinverse.py`:

```
modinverse.py

1 def egcd(a, b):
2     if b == 0:
3         return a, 1, 0
4     g, x, y = egcd(b, a % b)
5     return g, y, x - (a // b) * y
6
7 def mod_inverse(a, m):
8     g, x, _ = egcd(a, m)
9     return x % m if g == 1 else None
10
11 print('inverse of 3 mod 26 =', mod_inverse(3, 26))
12 print('check =', (3 * mod_inverse(3, 26)) % 26)
```

Run it and check the output.



```
Command Prompt

C:\crypto> python modinverse.py
inverse of 3 mod 26 = 9
check (a * inv) % m = 1
```

The inverse of 3 modulo 26 is 9, and $(3 \cdot 9) \bmod 26 = 1$.

Explanation. Since $3 \cdot 9 = 27 = 26 + 1$, we have $(3 \cdot 9) \bmod 26 = 1$, so 9 is the inverse. This is exactly how RSA finds its private key from its public key.

6.4 Euler's Totient

Task 6.4 ■ In-Lab — Compute Euler's totient

Write a program `totient.py` that computes $\varphi(n)$ by counting how many numbers from 1 to n have $\gcd(n, k) = 1$. Use it to find $\varphi(26)$ and $\varphi(33)$. What do you notice about $\varphi(33)$ compared with $(3 - 1) \times (11 - 1)$? We will explore this together in the lab.

Task 6.5 ■ Assignment — Prepare an RSA key

Using your functions, take the primes $p = 61$ and $q = 53$. Compute $n = pq$ and $\varphi = (p - 1)(q - 1)$, then find the private exponent d as the modular inverse of the public exponent $e = 17$ modulo φ . Report n , φ , and d , and check that $(e \cdot d) \bmod \varphi = 1$.

Task 6.6 ■ Assignment — A first taste of RSA

Using the values from Task 6.5 ($n = 3233$, $e = 17$), “encrypt” the number $m = 65$ by computing $c = m^e \bmod n$ with `pow`. Verify your result with the **Modular Power** tool in the [Cipher Playground](#) (base 65, exponent 17, modulus 3233). Then “decrypt” by computing $c^d \bmod n$ with $d = 2753$ and confirm you get 65 back.

6.5 Key Terms

Key Terms

Modulo The remainder after division; the basis of all these operations.

GCD The greatest common divisor; found with the Euclidean algorithm.

Coprime Two numbers whose GCD is 1.

Modular inverse The number x with $(a \cdot x) \bmod m = 1$; found with the Extended Euclidean algorithm.

Euler’s totient $\varphi(n)$ The count of numbers from 1 to n that are coprime to n .

LAB 7

Symmetric Encryption: DES and AES

The classical ciphers of the earlier labs are easy to break with a computer. Real systems use modern **block ciphers** — DES and, today, AES — that encrypt fixed-size blocks of bytes with a shared secret key. These algorithms are far too complex to implement by hand, so we use a trusted library, **pycryptodome**, exactly as real software does.

Lab Objectives

- Encrypt and decrypt data with AES using the pycryptodome library.
- Use a secure mode of operation (CBC) with an initialisation vector.
- See why ECB mode is unsafe.
- Use the older DES cipher and understand why AES replaced it.

Background *(the full concept is covered in the lecture)*

A **block cipher** encrypts a fixed-size block of bytes at a time (8 bytes for DES, 16 for AES) under a secret key. **DES** uses a 56-bit key, which is now small enough to brute-force, so it was replaced by **AES** (128-, 192-, or 256-bit keys). A **mode of operation** decides how successive blocks are handled: **ECB** encrypts each block independently (unsafe), while **CBC** chains each block to the previous one using an **initialisation vector** (IV). The lecture covers the internal structure of these ciphers (course ebook, Chapter 5); here we use them correctly.

Watch Out

This lab needs the pycryptodome library. Install it once with:

```
$ pip install pycryptodome
```

DES and AES are not in the Cipher Playground, so you verify your work the same way as in Lab 5: encrypt a message, then decrypt the result and check you get the original back.

7.1 AES in CBC Mode

Task 7.1 ■ Worked — Encrypt and decrypt with AES-CBC

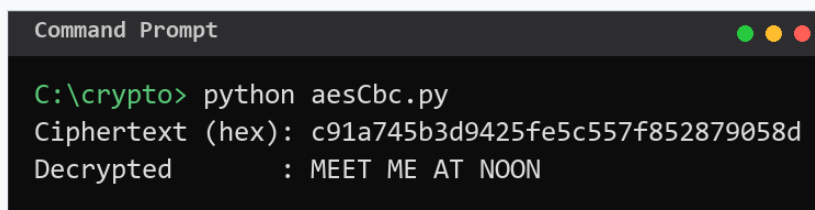
Encrypt MEET ME AT NOON with AES-128 in CBC mode, then decrypt it back.

Approach. AES needs a key (16 bytes for AES-128) and CBC needs a 16-byte IV. The message is padded to a whole number of 16-byte blocks, encrypted, and then decrypted with the same key and IV.

Code. Save this as `aesCbc.py`:

```
                                aesCbc.py
1  from Crypto.Cipher import AES
2  from Crypto.Util.Padding import pad, unpad
3
4  key = b'0123456789abcdef'      # 16 bytes = AES-128
5  iv  = b'abcdef9876543210'      # 16-byte initialisation vector
6  message = b'MEET ME AT NOON'
7
8  cipher = AES.new(key, AES.MODE_CBC, iv)
9  ciphertext = cipher.encrypt(pad(message, AES.block_size))
10 print('Ciphertext (hex):', ciphertext.hex())
11
12 decipher = AES.new(key, AES.MODE_CBC, iv)
13 plaintext = unpad(decipher.decrypt(ciphertext), AES.block_size)
14 print('Decrypted      :', plaintext.decode())
```

Run it and check the output.



```
Command Prompt
C:\crypto> python aesCbc.py
Ciphertext (hex): c91a745b3d9425fe5c557f852879058d
Decrypted      : MEET ME AT NOON
```

AES-CBC encrypts the message to 16 bytes (shown in hex) and decrypts it back to MEET ME AT NOON.

Explanation. The ciphertext is raw bytes, so we print it in hexadecimal. Decryption uses the *same* key and IV, then `unpad` removes the padding to recover the exact original message.

Watch Out

The key and IV are fixed here only so the printout is repeatable. In real software you must use fresh random bytes, for example `from Crypto.Random import get_random_bytes` and `key = get_random_bytes(16)`.

7.2 Why ECB Mode Is Unsafe

Task 7.2 ■ Worked — See the ECB weakness

Show that ECB mode encrypts two identical plaintext blocks to two identical ciphertext blocks.

Approach. Encrypt a message made of two identical 16-byte blocks in ECB mode, then compare the two ciphertext blocks.

Code. Save this as `aesEcb.py`:

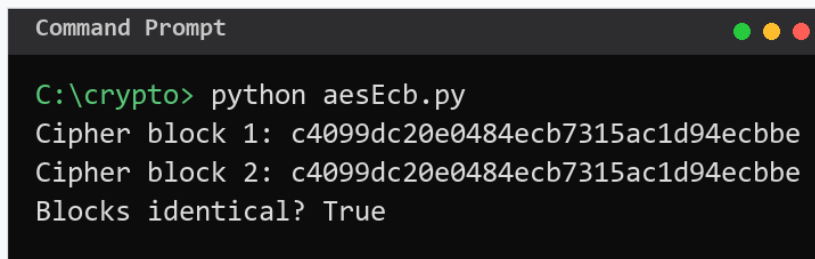
```
                                aesEcb.py
1  from Crypto.Cipher import AES
```

```

2 from Crypto.Util.Padding import pad
3
4 key = b'0123456789abcdef'
5 message = b'SAME BLOCK 12345SAME BLOCK 12345' # two identical blocks
6
7 cipher = AES.new(key, AES.MODE_ECB)
8 ciphertext = cipher.encrypt(pad(message, AES.block_size))
9 print('Cipher block 1:', ciphertext[0:16].hex())
10 print('Cipher block 2:', ciphertext[16:32].hex())
11 print('Blocks identical?', ciphertext[0:16] == ciphertext[16:32])

```

Run it and check the output.



```

Command Prompt

C:\crypto> python aesEcb.py
Cipher block 1: c4099dc20e0484ecb7315ac1d94ecbbe
Cipher block 2: c4099dc20e0484ecb7315ac1d94ecbbe
Blocks identical? True

```

In ECB mode the two identical plaintext blocks produce the same ciphertext block — a pattern an attacker can see.

Explanation. Because ECB encrypts each block independently, repeated data stays visible in the ciphertext. CBC avoids this by mixing each block with the previous one, which is why Task 7.1 used CBC.

7.3 The DES Cipher

Task 7.3 ■ Worked — Encrypt with DES

Encrypt TOP SECRET with DES.

Approach. DES works just like AES in the library, but its key is exactly 8 bytes and its block size is 8 bytes.

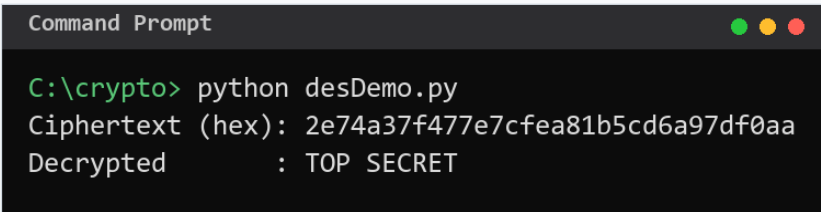
Code. Save this as `desDemo.py`:

```

desDemo.py
1 from Crypto.Cipher import DES
2 from Crypto.Util.Padding import pad, unpad
3
4 key = b'8bytekey' # DES key is exactly 8 bytes
5 message = b'TOP SECRET'
6
7 cipher = DES.new(key, DES.MODE_ECB)
8 ciphertext = cipher.encrypt(pad(message, DES.block_size))
9 print('Ciphertext (hex):', ciphertext.hex())
10
11 decipher = DES.new(key, DES.MODE_ECB)
12 print('Decrypted      :', unpad(decipher.decrypt(ciphertext),
13                               DES.block_size).decode())

```

Run it and check the output.



```
Command Prompt
C:\crypto> python desDemo.py
Ciphertext (hex): 2e74a37f477e7cfea81b5cd6a97df0aa
Decrypted      : TOP SECRET
```

DES encrypts and decrypts TOP SECRET.

Explanation. DES still works, but its 56-bit key is small enough that modern computers can try every key. That is why DES is considered broken and AES is used instead.

Task 7.4 ■ In-Lab — AES-CBC on your own message

Using `aesCbc.py`, encrypt a message of your own choice with AES-CBC, print the ciphertext in hex, and then decrypt it to confirm you recover your message. We will run this together in the lab.

Task 7.5 ■ Assignment — Triple DES

Triple DES (3DES) applies DES three times with a longer key to fix DES's short-key weakness. Write a program `tripleDes.py` that encrypts and decrypts TOP SECRET with DES3 (use a 24-byte key; the library's `DES3.adjust_key_parity` prepares a valid key). Confirm the decrypted text matches the original.

Task 7.6 ■ Assignment — AES-256

Modify `aesCbc.py` to use AES-256 instead of AES-128 by using a 32-byte key (the rest of the code is unchanged). Encrypt and decrypt MEET ME AT NOON, and print the key length in bits to confirm it is 256.

7.4 Key Terms

Key Terms

Block cipher A cipher that encrypts fixed-size blocks of bytes under a key.

DES An older block cipher with a 56-bit key, now brute-forceable.

AES The modern block-cipher standard, with 128-, 192-, or 256-bit keys.

Mode of operation How successive blocks are combined (ECB, CBC, and others).

Initialisation vector (IV) A random starting block that makes CBC encryption unpredictable.

LAB 8

Public-Key Cryptography: RSA

Every cipher so far has been *symmetric*: the same secret key encrypts and decrypts, so both sides must somehow share that key in advance. RSA solves this with two different keys — a **public key** anyone can use to encrypt, and a **private key** only the owner has, to decrypt. In this lab you build a complete (small-number) RSA from the number theory of Lab 6.

Lab Objectives

- Generate an RSA key pair from two primes.
- Encrypt and decrypt a number with RSA.
- Encrypt and decrypt a short text message.
- Explain why RSA's security rests on the difficulty of factoring.

Background *(the full concept is covered in the lecture)*

RSA builds its keys from two primes p and q . It computes $n = pq$ and $\varphi = (p - 1)(q - 1)$, picks a public exponent e coprime to φ , and finds the private exponent d as the modular inverse of e modulo φ (all from Lab 6). The **public key** is (e, n) and the **private key** is (d, n) . Encryption and decryption are just modular exponentiation:

$$c = m^e \bmod n, \quad m = c^d \bmod n.$$

Its security comes from the fact that recovering d needs φ , which needs the factors p and q of n — and factoring a large n is infeasible. The lecture covers the full theory (course ebook, Chapter 6).

Note

We use tiny primes so the numbers are readable and you can verify them in the Cipher Playground. Real RSA uses primes hundreds of digits long, through libraries such as `pycryptodome`; the mathematics is exactly the same.

8.1 Generating a Key Pair

Task 8.1 ■ Worked — Generate an RSA key pair

Build an RSA key pair from the primes $p = 61$ and $q = 53$.

Approach. Compute n and φ , choose $e = 17$, and get d as the modular inverse of e modulo φ (the `mod_inverse` from Lab 6).

Code. Save this as `rsa.py`:

rsa.py

```

1 def egcd(a, b):
2     if b == 0:
3         return a, 1, 0
4     g, x, y = egcd(b, a % b)
5     return g, y, x - (a // b) * y
6
7 def mod_inverse(a, m):
8     g, x, _ = egcd(a, m)
9     return x % m if g == 1 else None
10
11 p, q = 61, 53
12 n = p * q                # 3233 (public)
13 phi = (p - 1) * (q - 1)  # 3120 (secret)
14 e = 17
15 d = mod_inverse(e, phi)  # 2753 (private)
16
17 print('Public key (e, n):', (e, n))
18 print('Private key (d, n):', (d, n))

```

Explanation. The public key (17, 3233) can be shared with anyone. The private key (2753, 3233) must stay secret. Only n is shared from the key generation; p , q , and φ are discarded or hidden.

8.2 Encrypting and Decrypting a Number

Task 8.2 ■ Worked — Encrypt and decrypt a number

Encrypt the number 65 with the public key, then decrypt it with the private key.

Approach. Encryption is $c = m^e \bmod n$ and decryption is $m = c^d \bmod n$, both with Python's `pow(base, exp, mod)` from Lab 6.

Code. Add to `rsa.py`:

rsa.py (added)

```

1 def encrypt(m):
2     return pow(m, e, n)
3
4 def decrypt(c):
5     return pow(c, d, n)
6
7 message = 65
8 ciphertext = encrypt(message)
9 print('Message   :', message)
10 print('Encrypted :', ciphertext)
11 print('Decrypted :', decrypt(ciphertext))

```

Run it and check the output.

```
Command Prompt

C:\crypto> python rsa.py
Public key (e, n): (17, 3233)
Private key (d, n): (2753, 3233)
Message    : 65
Encrypted  : 2790
Decrypted  : 65
```

RSA encrypts 65 to 2790 and decrypts it back to 65.

Explanation. Encrypting 65 gives $65^{17} \bmod 3233 = 2790$, and decrypting gives $2790^{2753} \bmod 3233 = 65$. The private exponent d perfectly undoes what e did.

Cipher Playground — Modular Power

Verify the encryption step in the [Cipher Playground](#) under **Crypto Calculations**: the **Modular Power** tool with base 65, exponent 17, modulus 3233 gives Result = 2790.

8.3 Encrypting Text

Task 8.3 ■ Worked — Encrypt a short message

Encrypt the word HELLO with RSA by encrypting each letter's code number.

Approach. Turn each character into its numeric code with `ord`, encrypt that number, and store the list. To decrypt, raise each number to d and turn it back into a character with `chr`.

Code. Save this as `rsaText.py`:

```
rsaText.py

1 e, d, n = 17, 2753, 3233
2
3 message = 'HELLO'
4 ciphertext = [pow(ord(ch), e, n) for ch in message]
5 print('Message    :', message)
6 print('Ciphertext:', ciphertext)
7
8 decrypted = ''.join(chr(pow(c, d, n)) for c in ciphertext)
9 print('Decrypted  :', decrypted)
```

Run it and check the output.

Command Prompt

```
C:\crypto> python rsaText.py
Message    : HELLO
Ciphertext: [3000, 28, 2726, 2726, 1307]
Decrypted  : HELLO
```

Each letter of HELLO becomes a number; decrypting the list restores the word.

Explanation. Because every character code is smaller than $n = 3233$, each one encrypts independently. (Encrypting letter by letter like this is only for learning — real RSA encrypts a whole block at once and adds padding.)

Task 8.4 ■ In-Lab — Encrypt your own number

Using the key pair in `rsa.py`, encrypt the number 100 and then decrypt it to confirm you recover 100. Verify the encryption in the Cipher Playground (Modular Power, base 100, exponent 17, modulus 3233). We will do this together in the lab.

Task 8.5 ■ Assignment — Generate your own key

Pick two different primes, for example $p = 47$ and $q = 59$, and use the code in `rsa.py` to build a new key pair: compute n , φ , and d for $e = 17$. Report your public and private keys, then encrypt and decrypt a number of your choice to confirm the pair works.

Task 8.6 ■ Assignment — Why factoring matters

In two or three sentences, explain why an attacker who knows only the public key (e, n) cannot easily find the private exponent d . Mention what secret value d depends on, and why finding it requires factoring n .

8.4 Key Terms

Key Terms

Public-key cryptography Encryption that uses a public key to encrypt and a separate private key to decrypt.

Public key The pair (e, n) , shared with everyone, used to encrypt.

Private key The pair (d, n) , kept secret, used to decrypt.

RSA A public-key cipher whose security rests on the difficulty of factoring large numbers.

Factoring Finding the prime factors of n ; infeasible when n is large, which is what keeps RSA secure.

LAB 9

Cryptographic Hash Functions and HMAC

Encryption keeps a message secret. Sometimes, though, we need something different: a way to check that a message has not been *changed*, or to store a fingerprint of it. That is the job of a **cryptographic hash function**. In this lab you use SHA-256 and HMAC from Python's standard library — no extra packages needed.

Lab Objectives

- Compute a SHA-256 hash of a message with `hashlib`.
- Observe the avalanche effect: a tiny change flips the whole hash.
- Use a hash to detect tampering.
- Create a message authentication code with HMAC.

Background *(the full concept is covered in the lecture)*

A **cryptographic hash function** turns any input into a fixed-size **digest** (SHA-256 always gives 256 bits, i.e. 64 hex characters). Good hash functions are **one-way** (you cannot recover the input from the digest), **collision-resistant** (it is infeasible to find two inputs with the same digest), and show the **avalanche effect** (changing one bit of the input changes about half the output bits). An **HMAC** combines a hash with a secret key to make a **message authentication code** that proves both the integrity and the origin of a message. The lecture covers the theory (course ebook, Chapter 7).

9.1 Hashing with SHA-256

Task 9.1 ■ Worked — Compute a SHA-256 hash

Compute the SHA-256 digest of the word `hello`.

Approach. `hashlib.sha256` takes bytes, so we encode the string first, then ask for the digest as a hexadecimal string with `hexdigest`.

Code. Save this as `sha256demo.py`:

```
sha256demo.py

1 import hashlib
2
3 message = 'hello'
4 digest = hashlib.sha256(message.encode()).hexdigest()
5
6 print('Message:', message)
7 print('SHA-256:', digest)
```

Run it and check the output.

```
Command Prompt

C:\crypto> python sha256demo.py
Message: hello
SHA-256: 2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824
```

*The SHA-256 digest of **hello** is a 64-character hexadecimal string.*

Explanation. No matter how long the input is, the digest is always the same length. The same input always gives the same digest, but you cannot go backwards from the digest to the input.

Cipher Playground — Hash Laboratory

Verify it: open the [Cipher Playground](#), go to the **Hash Laboratory**, type **hello**, and press *Calculate SHA-256*. It should produce exactly the same 64-character digest as your program.

9.2 The Avalanche Effect

Task 9.2 ■ Worked — See the avalanche effect

Hash two words that differ by a single letter and compare the results.

Approach. Hash **hello** and **hellp** and print both digests.

Code. Save this as **avalanche.py**:

avalanche.py

```
1 import hashlib
2
3 for text in ['hello', 'hellp']:
4     print(text, '->', hashlib.sha256(text.encode()).hexdigest())
```

Run it and check the output.

```
Command Prompt

C:\crypto> python avalanche.py
hello -> 2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824
hellp -> fdd7585e08c4e2afd71dcabdb4636c89d557a3f42db9e2040c8bbd1708aa4ce7
```

Changing only the last letter produces a completely different digest.

Explanation. Even though the inputs differ by one letter, the two digests share almost nothing. This is the avalanche effect, and it is what makes a hash useful for detecting changes.

9.3 Message Authentication with HMAC

Task 9.3 ■ Worked — Create an HMAC

Produce an HMAC-SHA256 tag for a message using a secret key.

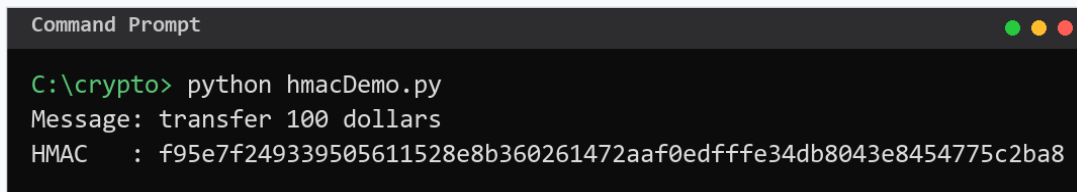
Approach. `hmac.new(key, message, hashlib.sha256)` combines the secret key with the message. Anyone with the same key can recompute the tag and check it; anyone without the key cannot forge it.

Code. Save this as `hmacDemo.py`:

`hmacDemo.py`

```
1 import hmac
2 import hashlib
3
4 key = b'secret'
5 message = b'transfer 100 dollars'
6
7 tag = hmac.new(key, message, hashlib.sha256).hexdigest()
8
9 print('Message:', message.decode())
10 print('HMAC   : ', tag)
```

Run it and check the output.



```
Command Prompt
C:\crypto> python hmacDemo.py
Message: transfer 100 dollars
HMAC   : f95e7f249339505611528e8b360261472aaf0edfffe34db8043e8454775c2ba8
```

The HMAC tag authenticates the message under the secret key.

Explanation. Unlike a plain hash, an HMAC depends on a secret key, so an attacker who changes the message cannot produce a matching tag without knowing the key. This proves the message is genuine and unchanged.

Task 9.4 ■ In-Lab — Detect tampering with a hash

Write a program `integrity.py` that computes the SHA-256 digest of the message `transfer 100 dollars`, then of a tampered version `transfer 900 dollars`, and prints whether the two digests match. What does the result tell you about using hashes to detect changes? We will run this together in the lab.

Task 9.5 ■ Assignment — Hash your own message

Use `sha256demo.py` to compute the SHA-256 digest of your full name. Verify it against the **Hash Laboratory** in the [Cipher Playground](#) using the same text, and confirm the two digests are identical.

Task 9.6 ■ Assignment — Authenticate with your own key

Modify `hmacDemo.py` to use your own secret key and your own message, and print the HMAC tag. Then, in one or two sentences, explain why a website should store the *hash*

of each user's password instead of the password itself.

9.4 Key Terms

Key Terms

Hash function A one-way function that turns any input into a fixed-size digest.

Digest The fixed-size output of a hash function (SHA-256 gives 64 hex characters).

Collision resistance The property that two different inputs are extremely unlikely to share a digest.

Avalanche effect A small change in the input changes about half of the output bits.

HMAC A keyed hash used as a message authentication code, proving integrity and origin.

LAB A

Instructor Solutions

This appendix gives solutions to the **In-Lab** and **Assignment** tasks of each lab. (The **Worked** tasks are solved in full inside the labs themselves.) It is for the instructor and teaching assistant; remove it before distributing the student edition.

Lab 1 — Setting Up Python and the Caesar Cipher

Solution 1.3 (In-Lab)

A brute-force program decrypts the ciphertext with every key and prints all candidates:

caesarBrute.py

```
1 SYMBOLS = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ'
2
3 def decrypt(message, key):
4     result = ''
5     for symbol in message.upper():
6         if symbol in SYMBOLS:
7             index = SYMBOLS.find(symbol)
8             result = result + SYMBOLS[(index - key) % 26]
9         else:
10            result = result + symbol
11    return result
12
13 ciphertext = 'KHOOR ZRUOG'
14 for key in range(1, 26):
15     print('Key', key, ':', decrypt(ciphertext, key))
```

```
Command Prompt

C:\crypto> python caesarBrute.py
Key 1 : JGNNQ YQTNF
Key 2 : IFMMP XPSME
Key 3 : HELLO WORLD
Key 4 : GDKKN VNQKC
Key 5 : FCJJM UMPJB
Key 6 : EBIL TLOIA
Key 7 : DAHHK SKNHZ
Key 8 : CZGGJ RJMGY
Key 9 : BYFFI QILFX
Key 10 : AXEEH PHKEW
Key 11 : ZWDDG OGJDV
Key 12 : YVCCF NFICU
Key 13 : XUBBE MEHBT
Key 14 : WTAAD LDGAS
Key 15 : VSZZC KCFZR
Key 16 : URYBY JBEPY
Key 17 : TQXXA IADXP
Key 18 : SPWWZ HZCWO
Key 19 : ROVVY GYBVN
Key 20 : QNUUX FXAUM
Key 21 : PMTTW EWZTL
Key 22 : OLSSV DVYSK
Key 23 : NKRRU CUXRJ
Key 24 : MJQQT BTWQI
Key 25 : LIPPS ASVPH
```

Every key tried. Key 3 gives readable text, HELLO WORLD, so the key is 3.

Solution 1.4 (Assignment)

Reading the message and key from the user:

caesarInput.py

```
1 SYMBOLS = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ'
2
3 def encrypt(message, key):
4     result = ''
5     for symbol in message.upper():
6         if symbol in SYMBOLS:
7             index = SYMBOLS.find(symbol)
8             result = result + SYMBOLS[(index + key) % 26]
```

```
9         else:
10             result = result + symbol
11         return result
12
13 message = input('Message: ')
14 key = int(input('Key: '))
15 print('Ciphertext:', encrypt(message, key))
```

Encrypting the name MINA with key 7 gives TPUH.

Solution 1.5 (Assignment)

Running the brute-force program on WKLV LV IXQ shows that key 3 produces the readable message THIS IS FUN. So the plaintext is THIS IS FUN and the key is 3.

Lab 2 — Monoalphabetic Substitution and Frequency Analysis

Solution 2.3 (In-Lab)

A program that counts and sorts the letters:

frequency.py

```
1 SYMBOLS = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ'
2 ciphertext = 'ZIT STZZTK T OL ZIT DGLZ EGDDGF STZZTK OF ' \
3             'TFUSOLI ZTBZ QFR YKTJXTFEN QFQSNLOL XLTL ZIOL YQEZ'
4
5 counts = {}
6 for symbol in ciphertext.upper():
7     if symbol in SYMBOLS:
8         counts[symbol] = counts.get(symbol, 0) + 1
9
10 for letter, count in sorted(counts.items(), key=lambda pair: pair[1], reverse=True)
11     :
12     print(letter, '=', count)
```

```
Command Prompt
C:\crypto> python frequency.py
T = 12
Z = 11
L = 8
F = 6
O = 5
I = 4
S = 4
Q = 4
K = 3
D = 3
G = 3
E = 3
Y = 2
X = 2
N = 2
U = 1
B = 1
R = 1
J = 1
```

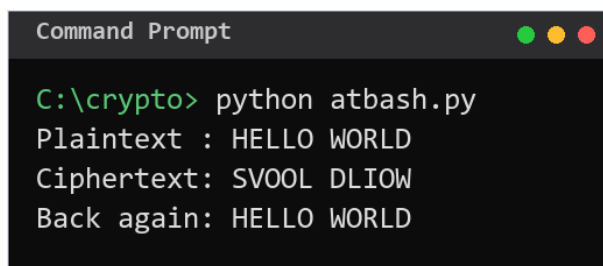
The most frequent letter is T (12 times), which most likely represents the English letter E.

Solution 2.4 (Assignment)

The Atbash program:

atbash.py

```
1 SYMBOLS = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ'
2
3 def atbash(message):
4     result = ''
5     for symbol in message.upper():
6         if symbol in SYMBOLS:
7             result = result + SYMBOLS[25 - SYMBOLS.find(symbol)]
8         else:
9             result = result + symbol
10    return result
11
12 print('Ciphertext:', atbash('HELLO WORLD'))
```



```

Command Prompt

C:\crypto> python atbash.py
Plaintext : HELLO WORLD
Ciphertext: SVOOL DLIOW
Back again: HELLO WORLD

```

Atbash turns HELLO WORLD into SVOOL DLIOW; applying it again returns the original, since Atbash is its own inverse.

Solution 2.5 (Assignment)

The most frequent English letters are E then T. Since T is the most frequent ciphertext letter and Z the second, the best first guess is $T \rightarrow E$ and $Z \rightarrow T$. (This is in fact correct for this ciphertext, whose plaintext begins THE LETTER E) Frequency analysis gives strong guesses for the most common letters; the rarer ones are then filled in from context.

Lab 3 — Transposition Ciphers: Rail Fence and Columnar

Solution 3.3 (In-Lab)

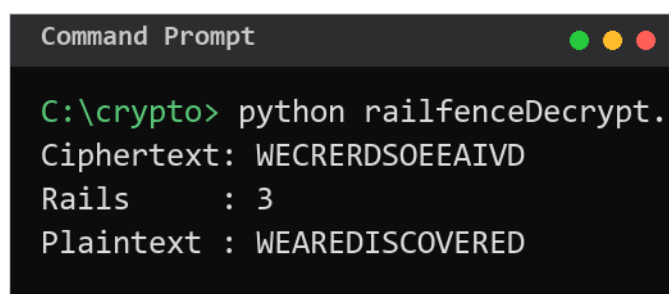
Rail Fence decryption first counts how many letters fall on each rail, slices the ciphertext accordingly, then walks the zigzag again to read them back in order:

```

railfenceDecrypt.py

1 def rail_pattern(length, rails):
2     pattern, row, step = [], 0, 1
3     for _ in range(length):
4         pattern.append(row)
5         if row == 0:
6             step = 1
7         elif row == rails - 1:
8             step = -1
9         row = row + step
10    return pattern
11
12 def decrypt(ciphertext, rails):
13     pattern = rail_pattern(len(ciphertext), rails)
14     counts = [pattern.count(r) for r in range(rails)]
15     fence, start = [], 0
16     for c in counts:
17         fence.append(list(ciphertext[start:start + c]))
18         start = start + c
19     result, position = '', [0] * rails
20     for r in pattern:
21         result = result + fence[r][position[r]]
22         position[r] = position[r] + 1
23     return result
24
25 print('Plaintext :', decrypt('WECRERDSOEAAIVD', 3))

```



```

Command Prompt

C:\crypto> python railfenceDecrypt.py
Ciphertext: WECRERDSOEEAIVD
Rails      : 3
Plaintext  : WEAREDISCOVERED

```

The ciphertext *WECRERDSOEEAIVD* decrypts back to *WEAREDISCOVERED*.

Solution 3.4 (Assignment)

With 4 rails, *WEAREDISCOVERED* encrypts to *WIREDSEEAECVDRO*. Changing the number of rails changes the zigzag path, so the letters are collected in a different order and the ciphertext changes.

Solution 3.5 (Assignment)

Answers vary with the chosen keyword. For example, *ATTACKATDAWN* with the keyword *SECRET* gives *TD TTCWAAAAKN*. The student's ciphertext should match the Cipher Playground's output for the same message and keyword.

Lab 4 — The Vigenere Cipher and the One-Time Pad

Solution 4.3 (In-Lab)

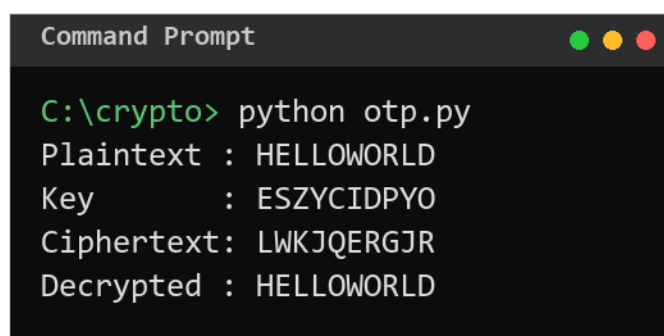
A one-time pad generates a random full-length key, then encrypts and decrypts like a Vigenere cipher:

otp.py

```

1 import random
2 SYMBOLS = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ'
3
4 def make_key(length):
5     return ''.join(random.choice(SYMBOLS) for _ in range(length))
6
7 def encrypt(message, key):
8     return ''.join(
9         SYMBOLS[(SYMBOLS.find(m) + SYMBOLS.find(k)) % 26]
10        for m, k in zip(message, key))
11
12 def decrypt(message, key):
13     return ''.join(
14         SYMBOLS[(SYMBOLS.find(m) - SYMBOLS.find(k)) % 26]
15        for m, k in zip(message, key))
16
17 message = 'HELLOWORLD'
18 key = make_key(len(message))
19 cipher = encrypt(message, key)
20 print('Key      :', key)
21 print('Ciphertext:', cipher)
22 print('Decrypted  :', decrypt(cipher, key))

```



```

Command Prompt

C:\crypto> python otp.py
Plaintext : HELLOWORLD
Key       : ESZYCIDPYO
Ciphertext: LWKJQERGJR
Decrypted : HELLOWORLD

```

A one-time pad run: a random key, the ciphertext, and the recovered plaintext. (The key is random, so your key and ciphertext will differ each run.)

Solution 4.4 (Assignment)

Answers vary with the chosen keyword. For example, MEETMEATNOON with the keyword KEY gives WICDQCKXLYSL. The student's ciphertext must match the Cipher Playground's Vigenère output for the same message and keyword, and decrypting with the same keyword must return MEETMEATNOON.

Solution 4.5 (Assignment)

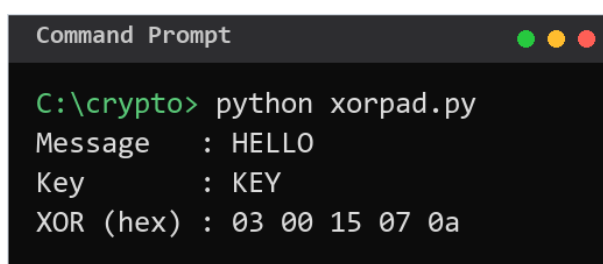
The XOR pad:

xorpad.py

```

1 def xor_hex(message, key):
2     out = []
3     for i, ch in enumerate(message):
4         out.append('{:02x}'.format(ord(ch) ^ ord(key[i % len(key)])))
5     return ' '.join(out)
6
7 print('XOR (hex) :', xor_hex('HELLO', 'KEY'))

```



```

Command Prompt

C:\crypto> python xorpad.py
Message   : HELLO
Key       : KEY
XOR (hex) : 03 00 15 07 0a

```

HELLO XOR KEY gives 03 00 15 07 0a, matching the XOR Demonstrator.

Reusing an XOR key is dangerous because XORing two ciphertexts made with the same key cancels the key and leaks the XOR of the two plaintexts, which can be separated with language patterns.

Lab 5 — The Playfair and Hill Ciphers

Solution 5.3 (In-Lab)

The keyword SECURITY gives the key square SECURITYABDFGHKLMNOPQVWXZ. Encrypting MEET ME AT NOON with playfair.py produces the ciphertext VTTFVTBYOPPO, and decrypting it returns MEETMEATNOON.

Solution 5.4 (Assignment)

Answers vary. The check is that decrypting the student's ciphertext with the same keyword returns their original message (a trailing padding X is acceptable). For example, keyword MONARCHY and message HIDE THE GOLD give BFCKPDFIMPBZ, which decrypts to HIDETHEGOLDX.

Solution 5.6 (Assignment)

Answers vary with the message. For example, CRYPTO with the key matrix $\begin{bmatrix} 3 & 3 \\ 2 & 5 \end{bmatrix}$ encrypts to FLNTVE. The grade depends on correct block-by-block matrix multiplication modulo 26.

Lab 6 — Mathematical Foundations of Cryptography**Solution 6.4 (In-Lab)**

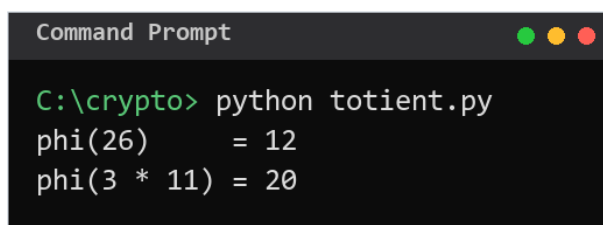
Euler's totient by counting coprime numbers:

totient.py

```

1 def gcd(a, b):
2     while b != 0:
3         a, b = b, a % b
4     return a
5
6 def totient(n):
7     return sum(1 for k in range(1, n + 1) if gcd(n, k) == 1)
8
9 print('phi(26)      =', totient(26))
10 print('phi(3 * 11) =', totient(33))

```



```

Command Prompt
C:\crypto> python totient.py
phi(26)      = 12
phi(3 * 11) = 20

```

$\varphi(26) = 12$ and $\varphi(33) = 20$.

Note that $\varphi(33) = 20 = (3 - 1) \times (11 - 1)$: for a product of two distinct primes p and q , $\varphi(pq) = (p - 1)(q - 1)$. This is the shortcut RSA relies on.

Solution 6.5 (Assignment)

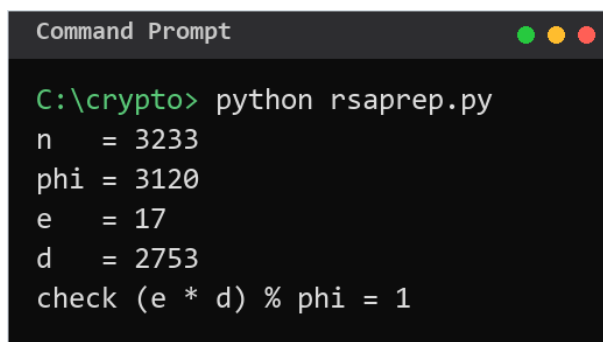
With $p = 61$ and $q = 53$: $n = 3233$ and $\varphi = 60 \times 52 = 3120$. The private exponent is $d = \text{mod_inverse}(17, 3120) = 2753$, and $(17 \times 2753) \bmod 3120 = 1$.

rsaprep.py

```

1 p, q = 61, 53
2 n = p * q
3 phi = (p - 1) * (q - 1)
4 e = 17
5 d = mod_inverse(e, phi)      # from modinverse.py
6 print('n =', n, ' phi =', phi, ' d =', d)

```



```

Command Prompt

C:\crypto> python rsaprep.py
n   = 3233
phi = 3120
e   = 17
d   = 2753
check (e * d) % phi = 1

```

RSA key preparation: $n = 3233$, $\varphi = 3120$, $d = 2753$.

Solution 6.6 (Assignment)

Encrypting: $c = 65^{17} \bmod 3233 = 2790$ (this matches the Cipher Playground's Modular Power tool). Decrypting: $2790^{2753} \bmod 3233 = 65$, the original number. In Python:

rsademo.py

```

1 n, e, d = 3233, 17, 2753
2 c = pow(65, e, n)           # 2790
3 print('cipher =', c)
4 print('back   =', pow(c, d, n)) # 65

```

Lab 7 — Symmetric Encryption: DES and AES

Solution 7.4 (In-Lab)

Any message works; the check is that decryption returns it exactly. Reusing `aesCbc.py` with a different message, for example:

aesCbc.py (message changed)

```

1 message = b'ATTACK AT DAWN'
2 cipher = AES.new(key, AES.MODE_CBC, iv)
3 ciphertext = cipher.encrypt(pad(message, AES.block_size))
4 print('Ciphertext (hex):', ciphertext.hex())
5 decipher = AES.new(key, AES.MODE_CBC, iv)
6 print('Decrypted      :', unpad(decipher.decrypt(ciphertext),
7                               AES.block_size).decode())

```

The decrypted output must equal the chosen message.

Solution 7.5 (Assignment)

Triple DES with a 24-byte key:

tripleDes.py

```

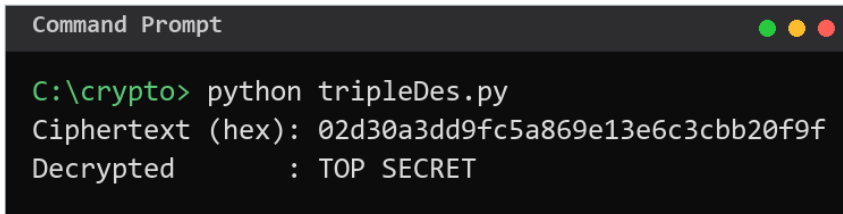
1 from Crypto.Cipher import DES3
2 from Crypto.Util.Padding import pad, unpad
3
4 key = DES3.adjust_key_parity(b'0123456789abcdef01234567') # 24 bytes
5 message = b'TOP SECRET'
6
7 cipher = DES3.new(key, DES3.MODE_ECB)
8 ciphertext = cipher.encrypt(pad(message, DES3.block_size))

```

```

9 print('Ciphertext (hex):', ciphertext.hex())
10
11 decipher = DES3.new(key, DES3.MODE_ECB)
12 print('Decrypted      :', unpad(decipher.decrypt(ciphertext),
13                               DES3.block_size).decode())

```



```

Command Prompt

C:\crypto> python tripleDes.py
Ciphertext (hex): 02d30a3dd9fc5a869e13e6c3cbb20f9f
Decrypted      : TOP SECRET

```

Triple DES encrypts TOP SECRET and decrypts it back.

Solution 7.6 (Assignment)

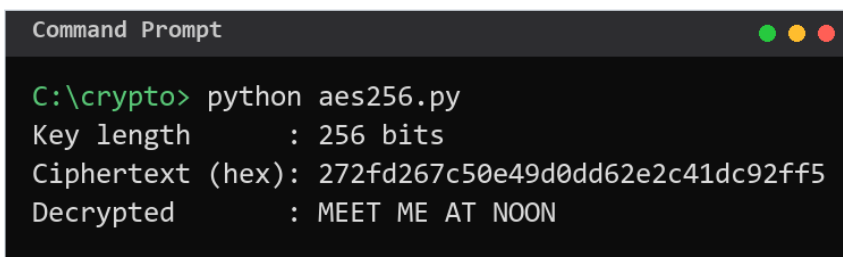
Only the key length changes — 32 bytes gives AES-256:

aes256.py

```

1 key = b'0123456789abcdef0123456789abcdef' # 32 bytes = AES-256
2 iv  = b'abcdef9876543210'
3 message = b'MEET ME AT NOON'
4
5 cipher = AES.new(key, AES.MODE_CBC, iv)
6 ciphertext = cipher.encrypt(pad(message, AES.block_size))
7 print('Key length      :', len(key) * 8, 'bits')
8 print('Ciphertext (hex):', ciphertext.hex())

```



```

Command Prompt

C:\crypto> python aes256.py
Key length      : 256 bits
Ciphertext (hex): 272fd267c50e49d0dd62e2c41dc92ff5
Decrypted      : MEET ME AT NOON

```

AES-256 uses a 32-byte key; the rest of the code is unchanged.

Lab 8 — Public-Key Cryptography: RSA

Solution 8.4 (In-Lab)

Encrypting 100 with the public key (17, 3233) gives $100^{17} \bmod 3233 = 1773$, and decrypting gives $1773^{2753} \bmod 3233 = 100$. The Cipher Playground's Modular Power tool with base 100, exponent 17, modulus 3233 also gives 1773.

Solution 8.5 (Assignment)

Answers vary with the primes. For $p = 47$ and $q = 59$: $n = 2773$, $\varphi = 46 \times 58 = 2668$, and $d = \text{mod_inverse}(17, 2668) = 157$. So the public key is (17, 2773) and the private key is (157, 2773). For example, encrypting 42 gives $42^{17} \bmod 2773 = 1466$, and $1466^{157} \bmod 2773 = 42$.

Solution 8.6 (Assignment)

Sample answer: The private exponent d is the modular inverse of e modulo φ , so computing d requires knowing $\varphi = (p - 1)(q - 1)$. But φ can only be found from the prime factors p and q of n . An attacker sees n but not its factors, and factoring a large n is computationally infeasible, so d stays out of reach.

Lab 9 — Cryptographic Hash Functions and HMAC

Solution 9.4 (In-Lab)

A tampering check:

integrity.py

```
1 import hashlib
2
3 def digest(text):
4     return hashlib.sha256(text.encode()).hexdigest()
5
6 original = 'transfer 100 dollars'
7 tampered = 'transfer 900 dollars'
8 print('match?', digest(original) == digest(tampered))
```

Command Prompt

```
C:\crypto> python integrity.py
original: 32c1b9d2a8aca5b60fcc1bb7e42bcd5d6cddbae266c828af95144997b1c26ac1
tampered: d7ff8f8abce658d74b821d0145f79d9bef110d3b75fef5464086d3dc6774ee15
match? False
```

Changing 100 to 900 gives a completely different digest, so `match?` is `False`.

The two digests do not match, so any change to the message is easy to detect: if the digest still matches, the message is intact.

Solution 9.5 (Assignment)

Answers vary with the name. The check is that the program's 64-character digest is byte-for-byte identical to the one the Hash Laboratory produces for the same text. (SHA-256 is deterministic, so the same input always gives the same digest.)

Solution 9.6 (Assignment)

The HMAC tag varies with the chosen key and message; any correct `hmac.new(key, message, hashlib.sha256).hexdigest()` is acceptable. Sample answer for the password part: a website should store only the hash of a password so that if its database is stolen, the attacker sees digests, not the real passwords, and cannot easily reverse them to log in.